

文章笔记

文章笔记：Improving Deep Agents with Harness Engineering

基于 LangChain 博客文章整理

2026 年 4 月 2 日

Harness Engineering

仅改动 Harness（系统提示词 + 工具 + 中间件），
不改动底层模型，
在 Terminal Bench 2.0 上从 Top 30 跃升至 Top 5。

deepagents-cli: 52.8% → 66.5% (+13.7pp)

文章作者/来源：LangChain

发布日期：2025

预计阅读时长：约 15 分钟

文章链接：<https://blog.langchain.com/improving-deep-agents-with-harness-engineering/>

项目主页：<https://github.com/hqh1025/ai-course-notes>

目录

1 引言：什么是 Harness Engineering	2
1.1 为什么 Harness 如此重要	2
1.2 本章小结	2
2 Trace 分析：数据驱动的改进循环	2
2.1 Trace Analyzer Skill	2
2.2 发现的常见失败模式	3
2.3 本章小结	3
3 Build & Self-Verify：让 Agent 学会自我检验	3
3.1 问题：模型的”第一印象偏见”	3
3.2 四阶段问题解决框架	4
3.3 PreCompletionChecklistMiddleware	4
3.4 本章小结	4
4 上下文工程：帮助 Agent 理解环境	5
4.1 Context Engineering 的核心理念	5
4.2 三类上下文注入	5
4.2.1 目录结构与工具发现	5
4.2.2 可测试性提示	5
4.2.3 时间预算管理	5
4.3 本章小结	6
5 循环检测与计划重审	6
5.1 Doom Loop 问题	6
5.2 LoopDetectionMiddleware	6
5.3 本章小结	7
6 Reasoning 预算分配策略	7
6.1 Reasoning Compute 的权衡	7
6.2 Reasoning Sandwich 策略	7
6.3 实验数据	7
6.4 本章小结	8
7 实验结果与改进路径	8
7.1 整体改进轨迹	8
7.2 跨模型适配性	8
7.3 本章小结	9
8 实践指南：构建高效 Agent Harness 的原则	9
8.1 原则一：代替 Agent 做上下文组装	9
8.2 原则二：激进地推动 Self-Verification	9
8.3 原则三：用 Trace 作为反馈信号	9

8.4	原则四：短期内检测并修复坏模式	10
8.5	原则五：为每个模型定制 Harness	10
8.6	本章小结	10
9	总结与延伸	10
9.1	核心收获	10
9.2	对 Agent 从业者的启示	11
9.3	未来展望	11
9.4	拓展阅读	11

1 引言：什么是 Harness Engineering

LangChain 团队在这篇博客中分享了一个重要发现：在不更换底层模型（GPT-5.2-Codex）的前提下，仅通过改进 Agent Harness（即围绕模型构建的系统层——包括系统提示词、工具选择和执行流程），就将其编程 Agent（deepagents-cli）在 Terminal Bench 2.0 基准测试上的得分从 52.8% 提升到 66.5%，排名从 Top 30 跃升至 Top 5。

这一结果的核心启示是：**模型能力的释放程度很大程度上取决于 Harness 的设计质量。**

Harness Engineering 的核心定义

Harness 的目标是将模型本身”参差不齐的智能”（spiky intelligence）塑造为适配目标任务的稳定能力。Harness Engineering 关注的是围绕模型构建的**系统工程**，包括系统提示词（System Prompt）、工具集合（Tools）、执行流（Execution Flow）等。优化目标包括任务完成率、Token 效率、延迟等。

1.1 为什么 Harness 如此重要

当前的大语言模型本质上是黑箱：我们难以直接理解其内部推理机制。但 Harness 设计者可以控制的是模型的**输入**（Prompt、上下文注入）和对模型**输出**的后处理（中间件、循环检测等）。通过精心设计这些外部机制，可以在不改变模型权重的情况下显著提升实际表现。

LangChain 的方法论很简单：

1. 用 Traces（LangSmith 中的执行追踪）理解 Agent 的失败模式
2. 针对失败模式，调整 Harness 的三个旋钮：System Prompt、Tools、Middleware
3. 迭代验证改进效果

Terminal Bench 2.0 简介

Terminal Bench 2.0 是当前评估 Agentic Coding 能力的标准基准测试，包含 89 个任务，涵盖机器学习、调试、生物信息学等领域。测试流程由 Harbor 编排：为每个任务启动沙盒环境（Daytona），与 Agent 交互，运行验证和评分脚本。所有 Agent 操作均存储在 LangSmith 中，包括延迟、Token 计数和成本等指标。

1.2 本章小结

Harness Engineering 的核心理念是：模型智能是”原材料”，Harness 是”加工工艺”。同一个模型在不同 Harness 下的表现可以天差地别。LangChain 通过 Trace 驱动的迭代改进，展示了一套系统化的 Harness 优化方法论。

2 Trace 分析：数据驱动的改进循环

2.1 Trace Analyzer Skill

LangChain 将 Trace 分析封装为一个可复用的 Agent Skill（Trace Analyzer），用于系统性地分析实验运行中的错误并生成改进建议。其工作流程如下：

1. 获取 Traces: 从 LangSmith 拉取实验运行的执行追踪数据
2. 并行错误分析: 派出多个子 Agent 并行分析不同 Trace 中的错误
3. 汇总综合: 主 Agent 综合所有子 Agent 的发现, 生成改进建议
4. 针对性修改: 人工审核 (可选) 后, 对 Harness 做针对性调整

与 Boosting 的类比

这种方法类似于机器学习中的 Boosting: 每轮训练 (Harness 迭代) 重点关注上一轮中模型犯错的样本 (失败的 Trace)。通过聚焦错误, 迭代改进能快速收敛到更好的性能。

2.2 发现的常见失败模式

通过 Trace 分析, LangChain 发现了以下 Agent 的常见失败模式:

表 1: Agent 常见失败模式及应对策略

失败模式	表现	应对策略
不自我验证	写完代码后只看一遍就提交	Build-Verify 循环
推理错误	逻辑链条出错	提升 Reasoning 预算
忽视任务指令	未严格遵循 Task Spec	强化 Prompt
超时	在次要问题上花费过多时间	时间预算管理
死循环	对同一文件反复小修改	LoopDetectionMiddleware
环境认知不足	不了解可用工具和目录结构	上下文注入

过拟合风险

在 Trace 驱动的改进中, 需要警惕**过拟合**: 针对某个特定任务做出的修改可能导致其他任务性能回退。人工审核步骤 (Step 3) 的价值在于判断改进是否具有泛化性, 而非仅仅修复个案。

2.3 本章小结

Trace 分析是 Harness Engineering 的”眼睛”。通过将 Trace 分析自动化并封装为 Skill, LangChain 实现了快速、可重复的改进循环。发现的六类典型失败模式 (不验证、推理错误、忽视指令、超时、死循环、环境认知不足) 构成了后续所有 Harness 改进的方向指引。

3 Build & Self-Verify: 让 Agent 学会自我检验

3.1 问题: 模型的”第一印象偏见”

当前模型存在一个显著的行为偏见: 倾向于接受自己写出的第一个看似合理的解决方案。Trace 分析中最常见的失败模式就是——Agent 写了一个方案, 重新读了一遍自己的代码, 认为”看起来没问题”, 然后直接提交。

这种行为缺少了软件工程中至关重要的一步：**测试**。

Self-Verification 的核心洞察

今天的模型是出色的“自我改进机器”（self-improvement machines）。Self-Verification 允许 Agent 通过运行时反馈在单次执行中自我改进。但模型天然**不会**主动进入 Build-Verify 循环——需要 Harness 显式引导。

3.2 四阶段问题解决框架

LangChain 在 System Prompt 中注入了一个结构化的问题解决流程：

1. **Planning & Discovery (规划与探索)**：阅读任务描述，扫描代码库，基于任务规格和验证方式制定初始计划
2. **Build (构建)**：按计划实现，同时构建测试——不仅测试 Happy Path，还要覆盖 Edge Cases
3. **Verify (验证)**：运行测试，完整阅读输出，**对照任务规格**（而非对照自己的代码）进行验证
4. **Fix (修复)**：分析错误，回溯原始规格，修复问题

Verify 阶段的关键区分

验证时必须**对照任务规格 (Task Spec)**，而非对照自己写的代码。Agent 常犯的错误是：对照自己的代码验证自己的输出——这是循环论证，无法发现偏离需求的问题。

3.3 PreCompletionChecklistMiddleware

除了 Prompt 引导，LangChain 还引入了一个确定性机制：**PreCompletionChecklistMiddleware**。这个中间件在 Agent 准备退出时拦截它，强制注入一个提醒，要求 Agent 对照 Task Spec 做一轮完整的验证。

这类似于所谓的 Ralph Wiggum Loop——通过 Hook 机制在 Agent 即将退出时强制其继续执行验证步骤。

确定性 vs 概率性控制

Harness 设计中存在两种控制策略：

- **概率性控制**：通过 Prompt 引导模型行为（模型可能遵循也可能忽略）
- **确定性控制**：通过 Middleware/Hook 强制执行（代码层面保证一定触发）

两者互补：Prompt 设定意图方向，Middleware 兜底确保关键步骤不被跳过。LangChain 在 Self-Verification 上同时使用了这两种策略。

3.4 本章小结

Self-Verification 是 LangChain 在 Harness 改进中收益最大的单项优化。核心是通过 Prompt 引导和 Middleware 兜底，迫使 Agent 从“写完就交”变为“写完-测试-对照规格验证-修复”的闭环。关键在于验证对象是**任务规格**而非 Agent 自身代码。

4 上下文工程：帮助 Agent 理解环境

4.1 Context Engineering 的核心理念

Harness Engineering 的一个核心职能是做好上下文的交付工程 (Delivery Mechanism for Context Engineering)。Agent 在未知环境中执行任务时，面临大量上下文缺失：目录结构是什么？有哪些工具可用？评估标准是什么？时间限制多少？

Context Engineering 核心原则

Agent 对其环境、约束和评估标准了解得越充分，就越能自主地高质量完成工作。Harness 工程师的职责本质是：**代替 Agent 准备和交付上下文，使 Agent 能自主完成工作。**

4.2 三类上下文注入

LangChain 实施了三类关键的上下文注入：

4.2.1 目录结构与工具发现

LocalContextMiddleware 在 Agent 启动时自动执行：

- 映射当前工作目录及其父目录、子目录的结构
- 运行 Bash 命令发现可用工具（如 Python 安装路径）
- 将发现结果注入 Agent 上下文

这一步至关重要，因为上下文发现和搜索**本身就容易出错**。让 Agent 自己去探索环境会引入不必要的错误风险。通过确定性注入，可以消除这一错误面。

4.2.2 可测试性提示

Agent 通常不知道自己的代码需要被程序化测试验证。LangChain 在 Prompt 中明确告知：

- 你的代码将被自动化测试脚本评估（类似于提交代码后的 CI）
- Task Spec 中提到的文件路径必须严格遵循
- 不仅要处理 Happy Path，还要考虑 Edge Cases

“Slop Buildup” 现象

如果不提示 Agent 写可测试的代码，Agent 倾向于只验证“一切正常”的路径 (Happy Path)，忽略边界条件。随着时间推移，这种松散的编码习惯会导致“Slop Buildup”——代码在表面上看起来能工作，但在严格测试下会大量失败。强制模型遵循测试标准是一个有效的反制策略。

4.2.3 时间预算管理

Agent 在时间估计方面表现很差。LangChain 注入时间预算警告 (Time Budget Warnings)，在剩余时间不足时提醒 Agent：

- 停止探索新方向

- 完成当前工作
- 转入验证阶段

虽然真实世界的编程通常没有严格时限,但在基准测试(和很多生产场景)中,如果不告知 Agent 时间约束,它会一直”探索”而无法收敛。

4.3 本章小结

Context Engineering 的核心思想是”不要让 Agent 去猜”。通过在 Harness 层面主动注入目录结构、工具信息、可测试性要求和时间约束,可以显著减少 Agent 因信息不足而犯错的概率。这是一种”为 Agent 做 Onboarding”的策略。

5 循环检测与计划重审

5.1 Doom Loop 问题

Agent 一旦确定了某个方案,往往会表现出严重的**近视行为** (Myopic Behavior): 即使方案明显不可行,仍然反复对同一个文件做微小变种修改,形成所谓的”Doom Loop”。在 Trace 分析中,LangChain 观察到有些 Agent 在同一个错误方案上尝试了超过 10 次微调。

Doom Loop 的本质

Doom Loop 反映了模型的一个深层问题: 当模型对自己的方案产生了”信念锁定”,它会倾向于在**执行层面**做微调(调参数、改小细节),而不是在**策略层面**重新审视方案是否正确。这类似于人类的”沉没成本谬误”。

5.2 LoopDetectionMiddleware

LangChain 设计了 LoopDetectionMiddleware 来应对这一问题:

- **监控机制:** 通过 Tool Call Hook 追踪每个文件的编辑次数
- **触发条件:** 当同一文件的编辑次数超过阈值 N 时触发
- **干预方式:** 向 Agent 注入上下文信息,如”你已经对这个文件修改了多次,请考虑重新审视你的整体方案”

需要注意的是,这种干预是**建议性**的,而非强制性的——如果模型仍然认为当前方向正确,它可能会继续沿着原路走。

Harness 护栏的时效性

LangChain 团队特别强调: LoopDetectionMiddleware 是一种针对**当前模型缺陷**的设计启发式 (Design Heuristic)。随着模型能力的提升,这类护栏很可能会变得不再必要。但在当下,它们是帮助 Agent 正确、自主执行的有效工具。Harness 工程师需要围绕**当前模型的短板**做设计,同时为**更强模型**做好演进规划。

5.3 本章小结

Doom Loop 是当前 Agent 最顽固的失败模式之一。LoopDetectionMiddleware 通过追踪文件编辑频率并注入重审提示来缓解这一问题。这类“护栏”设计具有时效性——它们是为当前模型的短板量身定制的临时方案，随着模型进化将逐渐退役。

6 Reasoning 预算分配策略

6.1 Reasoning Compute 的权衡

Reasoning Model 可以自主运行数小时，因此一个关键决策是：**在每个子任务上分配多少推理计算量？**

GPT-5.2-Codex 提供了四档 Reasoning 模式：Low、Medium、High、XHigh。更高的 Reasoning 预算意味着：

表 2: Reasoning 预算的权衡

维度	高 Reasoning 预算	低 Reasoning 预算
推理质量	更深入的分析，更少的逻辑错误	可能遗漏复杂依赖关系
Token 消耗	可达 2 倍以上	更经济
时间消耗	更慢，可能触发超时	更快
适用阶段	规划、验证	常规实现

6.2 Reasoning Sandwich 策略

LangChain 实验发现了一个有趣的启发式——**Reasoning Sandwich**（推理三明治）：

Reasoning Sandwich: XHigh-High-XHigh

- **规划阶段** (XHigh)：充分理解问题，制定高质量计划。某些 Terminal Bench 任务非常复杂，好的计划能帮助更快到达可行解
- **实现阶段** (High)：按计划执行，不需要最高级别推理
- **验证阶段** (XHigh)：仔细检查结果，捕获错误，确保提交质量

这种“两头重、中间轻”的分配策略在实践中表现最优。

6.3 实验数据

LangChain 的实验结果揭示了一个反直觉的发现：

- 全程 XHigh：仅 53.9%（大量任务因超时失败）
- 全程 High：63.6%
- XHigh-High-XHigh Sandwich：66.5%

全程使用最高推理预算反而表现最差，因为 Reasoning 消耗的时间导致大量任务超时。这说明 Reasoning 预算并非越高越好，而是需要”把好钢用在刀刃上”。

Adaptive Reasoning 的未来方向

Claude 和 Gemini 系列模型已经开始支持 Adaptive Reasoning——让模型自行决定在每个步骤投入多少推理计算量。另一种思路是多模型协作架构：用大模型做规划，将实现工作交给小模型。这些都是 Harness Engineering 的前沿探索方向。

6.4 本章小结

Reasoning 预算分配是 Agent 性能的重要调节杠杆。全程最高预算会因超时反而降低性能。XHigh-High-XHigh 的”三明治”策略通过在规划和验证阶段投入更多推理资源，在实现阶段适度降低，实现了最优的性能-效率平衡。

7 实验结果与改进路径

7.1 整体改进轨迹

LangChain 的 Harness Engineering 改进过程可以总结为以下演进路径：

表 3: Harness 迭代改进路径

阶段	改进内容	得分	增幅
Baseline	默认 Prompt + 标准 Tools + 标准 Middleware	52.8%	—
+SV	加入 Self-Verification 四阶段框架 + PreCompletionChecklist	—	显著
+CE	加入 Context Engineering（目录、工具、时间预算）	—	中等
+LD	加入 LoopDetectionMiddleware	—	中等
+RS	采用 Reasoning Sandwich 策略	66.5%	—
总提升		+13.7 百分点	

值得注意的是，这 13.7 个百分点的提升完全来自 **Harness 改进**，模型（GPT-5.2-Codex）保持不变。

7.2 跨模型适配性

LangChain 还用早期版本的 Harness 测试了 Claude Opus 4.6，得分为 59.6%。这个分数具有竞争力，但低于针对 Codex 优化后的 Harness 的表现。原因并非 Claude 模型本身不行，而是**没有为 Claude 运行同样的 Harness 改进循环**。

Harness 并非模型通用

不同模型需要不同的 Prompting 策略 (参见 Codex 和 Claude 各自的 Prompting Guide)。虽然某些原则 (如重视验证、良好的上下文准备) 具有通用性, 但**针对每个模型跑几轮 Harness 迭代是必要的**。一个为 Model A 优化的 Harness 直接用于 Model B, 通常不会达到最优表现。

7.3 本章小结

实验结果清晰地展示了 Harness Engineering 的投资回报: 仅改动外部系统就能带来 13.7 个百分点的提升。但 Harness 改进并非一劳永逸——它需要针对特定模型做迭代优化, 不同模型需要不同的 Harness 配置。

8 实践指南: 构建高效 Agent Harness 的原则

基于 LangChain 的实验和 deepagents 的整体构建经验, 以下是构建 Agent Harness 的核心原则。

8.1 原则一: 代替 Agent 做上下文组装

当前 Agent 在陌生环境中的上下文组装能力还不成熟。主动注入以下信息可以大幅减少可避免的错误:

- 目录结构和可用工具
- 编码最佳实践
- 问题解决策略框架
- 评估标准和约束条件

8.2 原则二: 激进地推动 Self-Verification

模型对自己的第一个方案有天然的偏好。必须通过 Prompt 和 Middleware 双管齐下, 推动 Agent 主动验证:

- 运行测试
- 对照任务规格 (而非自身代码) 检查结果
- 迭代改进直到通过验证

8.3 原则三: 用 Trace 作为反馈信号

Trace 是 Agent 自我评估和调试的基础。关键是同时调试工具和推理——模型走错路的原因往往是缺少某个工具或不知道如何使用它。

8.4 原则四：短期内检测并修复坏模式

模型目前还不完美。Harness 设计者的职责是：

- 围绕当前模型的短板做设计（如循环检测、强制验证）
- 同时为更强模型规划演进路径
- 接受这些护栏最终会被淘汰——但今天它们是必要的

8.5 原则五：为每个模型定制 Harness

通用 Harness 是一个好的起点，但最优性能需要针对具体模型做迭代：

- 跑几轮 Trace 分析 → 改进循环
- 关注模型特有的失败模式
- 利用模型特有的优势

8.6 本章小结

五条原则构成了一个完整的 Harness Engineering 方法论：上下文注入降低环境不确定性、Self-Verification 确保输出质量、Trace 分析驱动迭代改进、护栏设计应对当前模型短板、模型专属定制最大化性能。

9 总结与延伸

9.1 核心收获

LangChain 这篇文章传递了一个强有力的信息：**在当前 AI Agent 的实践中，Harness Engineering 的边际收益可能远高于等待更强模型。**

关键数据支撑：

- 同一模型（GPT-5.2-Codex），仅改 Harness：52.8% → 66.5% (+13.7pp)
- 排名从 Top 30 跃升至 Top 5
- 改进手段全部是”系统工程”层面的——Prompt、Middleware、上下文注入

9.2 对 Agent 从业者的启示

Harness Engineering 的投资回报

在构建 Agent 应用时，不要只关注“用哪个模型”。更应该投入时间在：

- 设计 Trace 驱动的迭代改进流程
- 构建 Self-Verification 机制
- 做好 Context Engineering
- 识别并修补模型的行为短板

这些投入的性价比往往远高于简单地换一个更大/更贵的模型。

9.3 未来展望

LangChain 指出了几个值得关注的前沿方向：

1. **多模型系统**：让 Codex、Gemini、Claude 协同工作，各取所长
2. **持续学习的 Memory 原语**：让 Agent 能跨任务积累经验并自主改进
3. **跨模型 Harness 评估**：系统性地度量 Harness 变更对不同模型的影响
4. **RLM (Reinforcement Learning from Mistakes)**：更高效地从 Trace 中挖掘改进信号

此外，LangChain 已经开源了 Deep Agents（提供 Python 和 JavaScript 版本），并公开了实验 Trace 数据集供社区研究。

9.4 拓展阅读

- LangChain Blog 原文：Improving Deep Agents with Harness Engineering
- Deep Agents 开源仓库（Python & JavaScript）
- LangSmith：LangChain 的 Trace 和评估平台
- Terminal Bench 2.0：Agentic Coding 基准测试
- Harbor：Agent 评估编排工具